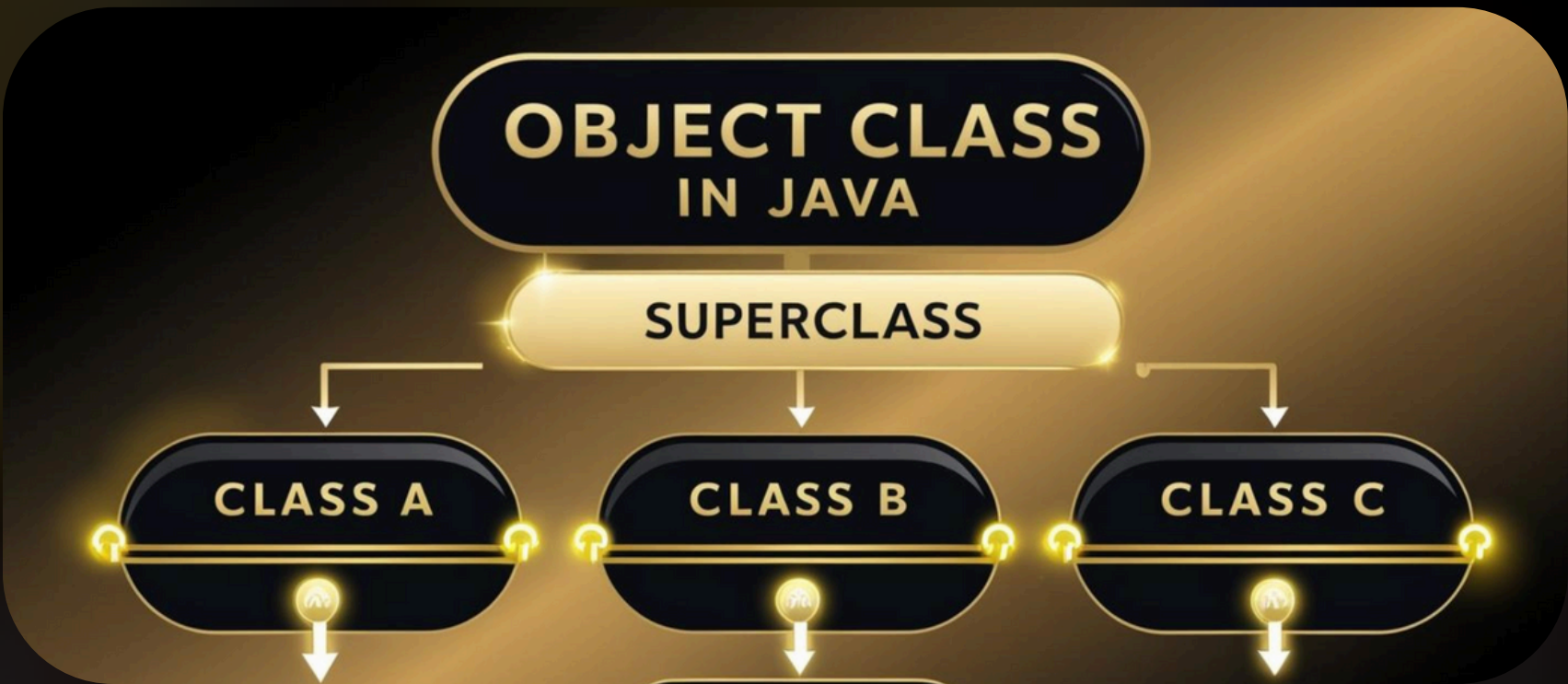




Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



OBJECT CLASS FAQ



What is Object class OR Default Super Class?

- By default, it is a super class for all the classes in java
- All the classes in java are the subclass of object class directly or indirectly
- The Object class defines some common behaviours which is applicable for all the objects
- Example: Cloning mechanism, comparing techniques, Garbage Collections, object can be notified
- Such as
 - Cloning mechanism: object can be compared
 - Object can be cloned
 - Object can be notified etc.
 - Comparing techniques
 - Garbage Collections
- All these features applicable for each and every object in java
- So, if we declare common functionality methods in object class It can be accessed by any java object

Important methods in Object class

- Object class has 9 non-static methods
- which can be accessed by any java class

Object class methods:

toString ()	Wait ()	Clone ()
Equals (Object obj)	Wait (long)	Finalize ()
hashCode ()	Wait (long, int)	getClass ()
	Notify ()	
	notifyAll ()	





Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



OBJECT CLASS METHODS FAQ

String toString () methods:

- Defined in object class
- **By default:** it returns the string representation of the object. [**Format:** Fullyqualifiedclassname@Address]
we can override the toString () method in the user defined class.
- Use: By overriding this we can get string representation of object

Why we need to override toString () method:

- By default: it returns the string representation of the object. [**Format:** Fullyqualifiedclassname@Address] and these details no use for end user
- Instead of displaying some garbage Address to end user we can override the toString () method in user defined class
- By overriding this we can get string representation of object
- We can display class members and its value to end user

Equals () Method

- Defined in Object class
- It is used to check the equality between any two objects
- Based on the hashCode number of objects, it will check the equality between objects
- It is used to compare the values of two objects
- if two objects having same value --- > It returns **TRUE**
- if two objects having different value --- > It returns **FALSE**

equals () v/s '==' Operator

- Both are used to check the equality between the two objects
- '==' operator is used for address comparison
- equals () method is used for content comparison

hashCode ()

- returns the hashCode number of the object
- If we invoke hashCode () methods on any object, it returns hashCode number of the object which is based on the hexadecimal address of the object
- We can change the default implementation of this method by overriding it
- But we should provide our own logic which generated unique hash code number

Why we need to override equals () and hashCode () method.

- "If two objects are equal using Object class equals method then the hashCode method should give the same value for these two objects."
- Hash code of 2 objects same when objects are equal using object class equal method'
- If we **override only equal** method, a. equals(b) are true
 - it means the **hashCode** of a and b **must** be same but not happen.
 - Note: **hashCode** () method of Object class always return new **hashCode** for each object.
 - So, when you **need** to use your object in the hashing-based collection, **must** **override** both **equals ()** and **hashCode ()**



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: @javatechcommunity.com - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



PURPOSE OF THIS DOCUMENT

- **Use this PDF as a quick last-minute revision guide only.**
- **For more details, please perform R&D and gain a deeper understanding of how memory architecture works.**
- **Once the interview series is complete, we will cover this topic in detail with code snippets.**



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com